

NAIJASHOP PROJECT

Ingestion Guide: Python vs. Airbyte Cloud

Datafrik Data Engineering Cohort

Goal: pull the live USD → NGN exchange rate into a database. This guide walks through two different ways to do it, using the exact same data source. Try both, and compare what each approach actually involves.

Before you start

This guide assumes the `ingest_exchange_rate.py` script (attached separately) and your NaijaShop Postgres database from Weeks 1–2 are already set up and working.

Part A — Ingestion with Python

What you need: the attached `ingest_exchange_rate.py` script, Python installed, and your NaijaShop Postgres running.

Step 1: Install the required libraries

```
pip install requests psycopg2-binary
```

Step 2: Run the script

```
python ingest_exchange_rate.py --dsn "dbname=naijashop user=postgres  
password=YOUR_PASSWORD host=localhost"
```

Replace `YOUR_PASSWORD` with your actual Postgres password.

Step 3: Confirm it worked

Expected output:

```
Fetching USD->NGN rate...  
1 USD = <number> NGN (as of ...)  
Loaded into exchange_rates table.
```

Step 4: Verify in pgAdmin or DBeaver

```
SELECT * FROM exchange_rates;
```

You should see one new row with the rate, a timestamp, and the source.

What just happened

Your own code called a live API, got back one number, and saved it to your database — full control, but you wrote every line of logic yourself.

Part B — Ingestion with Airbyte Cloud

What you need: an Airbyte Cloud account, and a free Neon Postgres account (neon.tech).

Why Neon instead of your local database

Airbyte Cloud runs in the cloud, so it cannot reach a database sitting on your own laptop. Neon gives you a free, cloud-hosted Postgres database that Airbyte Cloud CAN reach directly — no local exposure required.

Step 1: Sign up for Airbyte Cloud

Go to cloud.airbyte.com → Sign up. This is a 14-day free trial (400 credits), not a permanent free tier — treat this as a demo, not a production pipeline.

Step 2: Build the connector

1. Left sidebar → Builder → New custom connector → Start from scratch
2. Name it: NaijaShop Exchange Rate
3. Set base URL: <https://www.currencyexchangetool.com/api/v1>
4. Auth method: No Auth

Step 3: Add the stream

1. Click Add Stream → name it exchange_rate
2. URL path: /convert
3. Query parameters: amount=1, from=USD, to=NGN
4. Method: GET

Step 4: Test it

Click Test in the builder panel — you should see the live JSON response appear on the right (fields: rate, from, to, updatedAt).

Step 5: Publish the connector

Click Publish (top right) — this makes it usable in a real connection, not just the test sandbox.

Step 6: Set up your destination (Neon)

1. Sign up free at neon.tech, create a project
2. Copy the connection details Neon gives you
3. In Airbyte: Destinations → + New destination → Postgres → paste in Neon's details → Set up destination

Step 7: Create the connection

Connections → + New connection → pick your source and destination → set sync frequency to Daily → Set up connection.

Step 8: Run and verify

Click Sync now. Once it shows Succeeded, open Neon's SQL editor and run:

```
SELECT * FROM exchange_rate;
```

What just happened

No code written — a UI tool made the same API call and loaded the same kind of data, on a schedule, automatically.

Compare the Two Approaches

	Python	Airbyte Cloud
Code required	Yes — you write it	No — point and click
Cost	Free forever	Free trial only, then paid
Destination	Your own local Postgres	Cloud-only (e.g. Neon)
Best for	Learning how ingestion actually works	Seeing what a managed tool looks like

Assignment

Run both approaches, then write 2–3 sentences on which you'd choose for a real job, and why.